

13. Graphical User Interface (GUI)

Preface

GUIs give our applications an interface consisting of windows, menus, buttons, and other graphical components.

All GUI programming in Java is done through the use of dedicated standard class libraries. Once we understand the principles, we can find out all the necessary details by working with the standard library documentation. The principles we need to understand can be divided into three topic areas:

- What kinds of elements can we show on screen?
- How do we arrange those elements?
- How do we react to user input?

We shall discuss these questions under the keywords components, layout, and event handling.

Example 10.1

```
import javax.swing.*;
import java.awt.event.*;

public class SimpleGUI extends JFrame {
    // Fields for components
    private JLabel label;
    private JButton button;
    private JMenuBar menuBar;
    private JMenu menu;
    private JMenuItem menuItem;

    // Constructor to initialize the GUI
    public SimpleGUI() {

        // Set up the frame (window), these are method in the class "JFrame".
        setTitle("Simple GUI Example");
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Create and add JLabel
        label = new JLabel("Hello, World!");
        add(label, "North"); // Using BorderLayout by default

        // Create and add JButton
        button = new JButton("Click Me");
        add(button, "Center");

        // Set up JMenuBar
        menuBar = new JMenuBar();
        menu = new JMenu("File");
        menuItem = new JMenuItem("Exit");

        // JFrame <- menuBar <- menu <- menuItem
        menu.add(menuItem);
        menuBar.add(menu);
    }
}
```

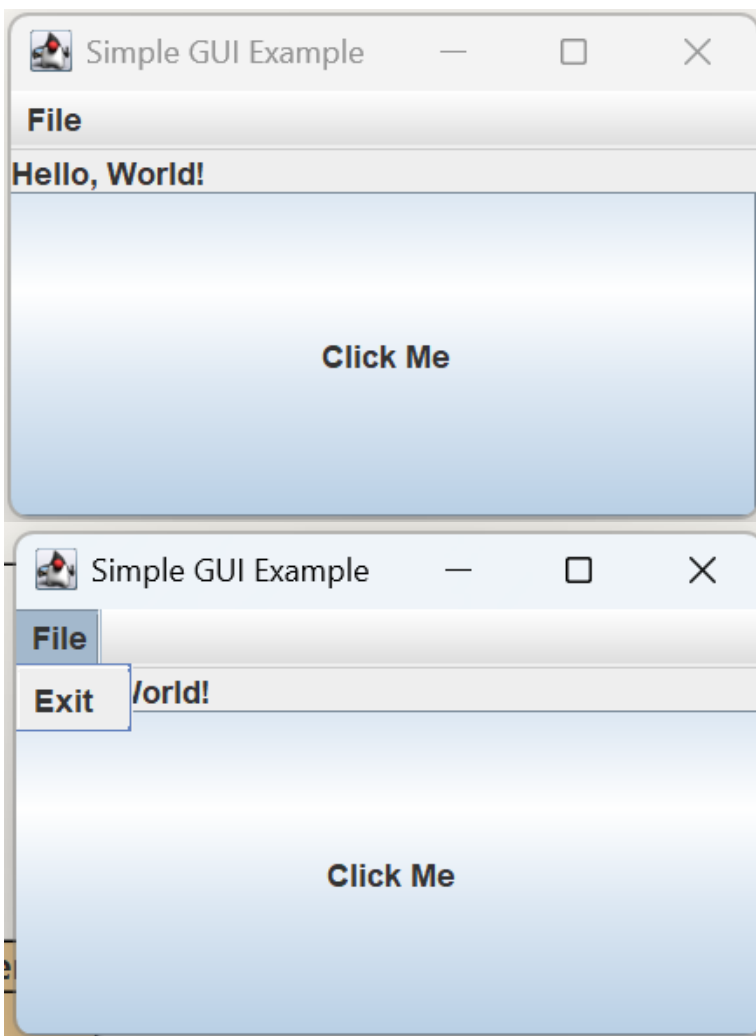
```

setJMenuBar(menuBar);

// Add action listener for button and menu item
ActionListener listener = new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == button) {
            label.setText("Button Clicked!");
        } else if (e.getSource() == menuItem) {
            System.exit(0);
        }
    }
};
button.addActionListener(listener);
menuItem.addActionListener(listener);
}

// Main method to run the application
public static void main(String[] args) {
    SimpleGUI gui = new SimpleGUI();
    gui.setVisible(true);
}
}

```



Example 10.2

```

import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;

/**
 * ImageViewer is the main class of the image viewer application. It builds
 * and displays the application GUI and initialises all other components.
 *
 * This application uses JavaFX. To start it, use the BlueJ
 * 'Run JavaFX Application' option in the class's context menu.
 *
 * @author Michael Kölling and David J. Barnes
 * @version 0.1
 */
public class ImageViewer extends Application
{
    /**
     * The main entry point for JavaFX programs.
     */
    @Override
    public void start(Stage stage)
    {
        Label myLabel = new Label("I am a label. I can display some text.");

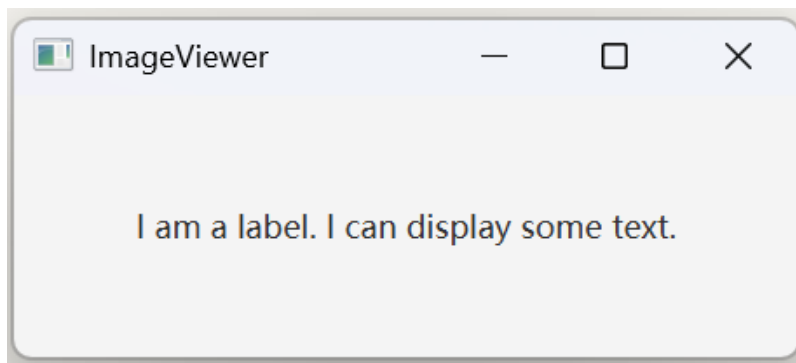
        StackPane root = new StackPane();
        root.getChildren().add(myLabel);

        Scene scene = new Scene(root, 300, 100);

        stage.setTitle("ImageViewer");
        stage.setScene(scene);
        stage.show();
    }
}

```

- The GUI is ...



Example 10.3

```

import javax.swing.*;
import java.awt.*;

public class LayoutExample extends JFrame {

```

```

// Constructor to initialize the GUI with different layouts
public LayoutExample() {
    setTitle("Layout Managers Example");
    setSize(400, 300);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // Panel with FlowLayout
    JPanel flowPanel = new JPanel(new FlowLayout());
    flowPanel.add(new JButton("Button 1"));
    flowPanel.add(new JButton("Button 2"));
    add(flowPanel, BorderLayout.NORTH);

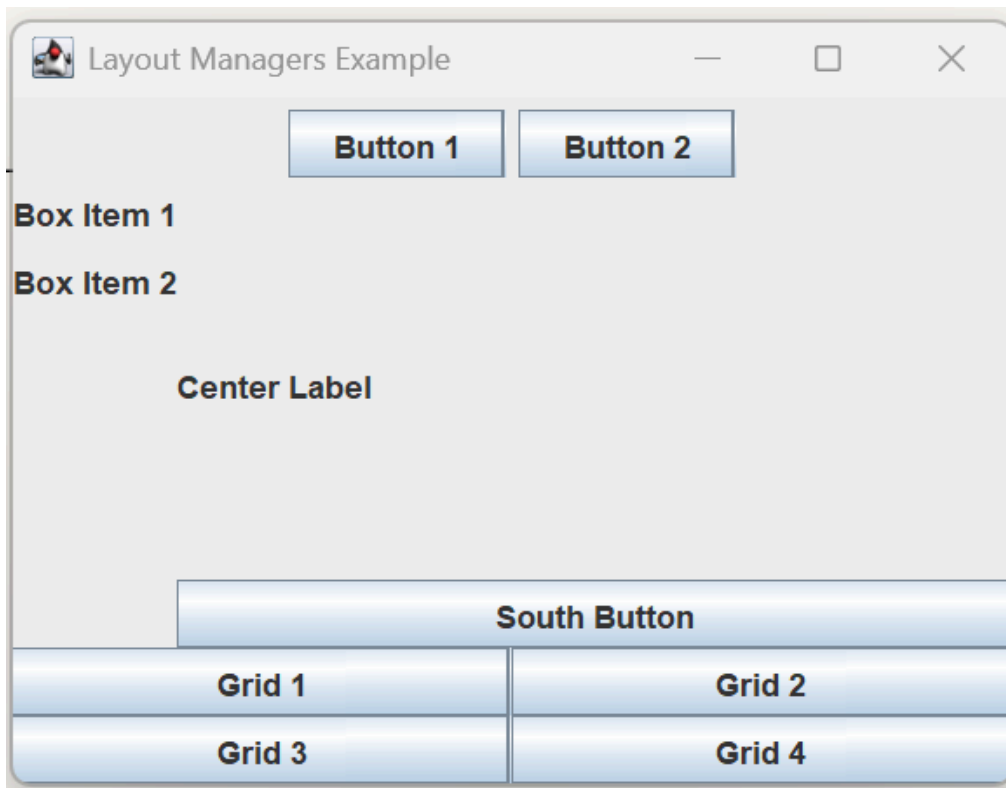
    // Panel with BorderLayout
    JPanel borderPanel = new JPanel(new BorderLayout());
    borderPanel.add(new JLabel("Center Label"), BorderLayout.CENTER);
    borderPanel.add(new JButton("South Button"), BorderLayout.SOUTH);
    add(borderPanel, BorderLayout.CENTER);

    // Panel with GridLayout (2 rows, 2 columns)
    JPanel gridPanel = new JPanel(new GridLayout(2, 2));
    gridPanel.add(new JButton("Grid 1"));
    gridPanel.add(new JButton("Grid 2"));
    gridPanel.add(new JButton("Grid 3"));
    gridPanel.add(new JButton("Grid 4"));
    add(gridPanel, BorderLayout.SOUTH);

    // Panel with BoxLayout (vertical)
    JPanel boxPanel = new JPanel();
    boxPanel.setLayout(new BoxLayout(boxPanel, BoxLayout.Y_AXIS));
    boxPanel.add(new JLabel("Box Item 1"));
    boxPanel.add(Box.createVerticalStrut(10)); // Spacer
    boxPanel.add(new JLabel("Box Item 2"));
    add(boxPanel, BorderLayout.WEST);
}

// Main method to run the application
public static void main(String[] args) {
    LayoutExample example = new LayoutExample();
    example.setVisible(true);
}
}

```



10.1. Graphical User Interface (GUI)

Definition

10.2. AWT and Swing

Definition

Java has three GUI libraries. The oldest is called AWT (Abstract Window Toolkit) and was introduced as part of the original Java API. Later, a much-improved GUI library, called Swing, was added to Java. More recently, the JavaFX library has been added.

Remark

- Wherever there are equivalent classes in AWT and Swing, the Swing versions have been identified by adding the letter J to the start of the class name. You will, for example, see classes named "Button" and "JButton", "Frame" and "JFrame", "Menu" and "JMenu", and so on.
 - JFrame, JLabel, JButton, JMenuBar, JMenu, JMenuItem: These are core classes from the Swing library (which builds on AWT). They represent the improved, J-prefixed versions of AWT components (e.g., JFrame vs. AWT's Frame). Swing uses some AWT elements but provides these for better GUI building.
-

10.3. Creating GUI Components

10.3.1. JFrame

Definition

JFrame is a top-level window container in Swing that represents the main window of a GUI application, providing a frame with title bar, borders, and controls like minimize, maximize, and close buttons.

Features

- Acts as the root container for other GUI components.
- Supports layouts for arranging components inside it.
- Can be resized, moved, and closed by the user.

Example

- Classes: Often extended or instantiated to create the main application window.

```
setTitle("Simple GUI Example");
setSize(300, 200);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

- Objects: The `SimpleGUI` frame in the example, which holds all other components.

```
// Starting JFrame in another class.
JFrame a = new JFrame("Simple GUI Example");
a.setSize(300,200);
a.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Remark

- In example 1.1, `SimpleGUI` extends `JFrame`, and methods like `setTitle()`, `setSize()`, and `setDefaultCloseOperation()` are used to configure it. Always call `setVisible(true)` to display it.

10.3.2. JLabel

Definition

JLabel is a Swing component used to display text, images, or both, typically for providing information or labels in a GUI without allowing user editing.

Features

- Displays non-editable text or icons.
- Can be aligned (left, center, right) and styled (font, color).
- Does not respond to user input by default.

Example

- In example 1.1, the `JLabel` is created as:

```
label = new JLabel("Hello, World!");
// I'll introduce "North" later.
add(label, "North");
```

Remark

- In example 1.1, the label's text is updated dynamically in response to events, showing how it can reflect changes in the application's state.
-

10.3.3. JButton

Definition

JButton is a Swing component that represents a clickable button in a GUI, used to trigger actions when pressed by the user.

Features

- Displays text or icons on the button.
- Generates an `ActionEvent` when clicked.
- Can be enabled/disabled and styled (e.g., background color).

Example

- In example 1.1, the `JButton` is created as:

```
button = new JButton("Click Me");  
add(button, "Center");
```

Remark

- In example 1.1, an `ActionListener` is added to the button to handle clicks, demonstrating how buttons are used for user interaction.
-

10.3.4. JMenuBar

Definition

JMenuBar is a Swing component that serves as a container for menus at the top of a `JFrame`, providing a menu bar like those in typical desktop applications.

Features

- Holds one or more `JMenu` objects.
- Attached to a `JFrame` using `setJMenuBar()`.
- Appears at the top of the window by default.

Example

- In example 1.1, the `JMenuBar` is created as:

```
menuBar = new JMenuBar();  
// ...
```

```
// Add menubar to JFrame.  
setJMenuBar(menuBar);
```

Remark

- In example 1.1, it's used to hold a single menu, but it can support multiple menus for more complex applications.
-

10.3.5. JMenu

Definition

JMenu is a Swing component that represents a pull-down menu in a menu bar, containing menu items or submenus.

Features

- Added to a JMenuBar.
- Can hold JMenuItem objects or other JMenus (for submenus).
- Displays a title and drops down when clicked.

Example

- In example 1.1, the JMenu is created as:

```
menu = new JMenu("File");  
// ...  
// Add menu to menuBar.  
menuBar.add(menu);
```

Remark

- In example 1.1, it contains a single JMenuItem, illustrating basic menu structure.
-

10.3.6. JMenuItem

Definition

JMenuItem is a Swing component that represents an individual selectable item in a menu, which can trigger an action when chosen.

Features

- Displays text or icons.
- Generates an Action Event when selected.
- Can be added to a JMenu.

Example

- In example 1.1, the JMenuItem is created as:

```
menuItem = new JMenuItem("Exit");  
// Add menuItem to menu.  
menu.add(menuItem);
```

Remark

- In example 1.1, it shares an ActionListener with the button, showing how menu items handle events similarly to buttons.
-

10.4. Layout

10.4.1. Layout Managers

Definition

Layout managers are objects in Java Swing that automatically control the size and position of GUI components within a container, handling dynamic aspects like resizing windows or different screen resolutions.

Features

- Provide flexible arrangement without fixed pixel coordinates.
- Adapt to changes such as font sizes, component additions, or user resizing.
- Used by setting them on containers like JPanel or JFrame via `setLayout()`.

Example

- In example 10.3, different panels use various layout managers to arrange buttons and labels automatically.

Remark

- Layout managers replace older coordinate-based systems, making GUIs more portable and responsive. Common types include FlowLayout, BorderLayout, GridLayout, and BoxLayout, each suited for different arrangements.
-

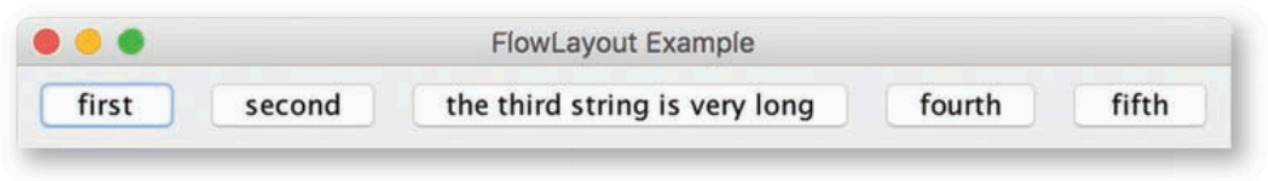
10.4.2. FlowLayout

Definition

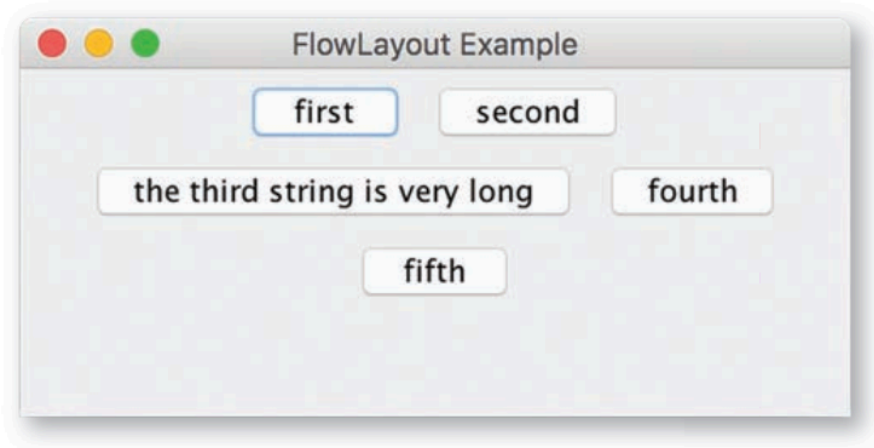
FlowLayout is a simple layout manager that arranges components in a row from left to right, wrapping to the next line if the container's width is exceeded, like words in a paragraph.

Features

- Figure 10.5



(a)



(b)

- Default layout for JPanel.
- Aligns components left, center (default), or right.
- Components retain their preferred sizes; no stretching.

Example

- In example 10.3, the flowPanel uses:

```
JPanel flowPanel = new JPanel(new FlowLayout());
flowPanel.add(new JButton("Button 1"));
flowPanel.add(new JButton("Button 2"));
```

Remark

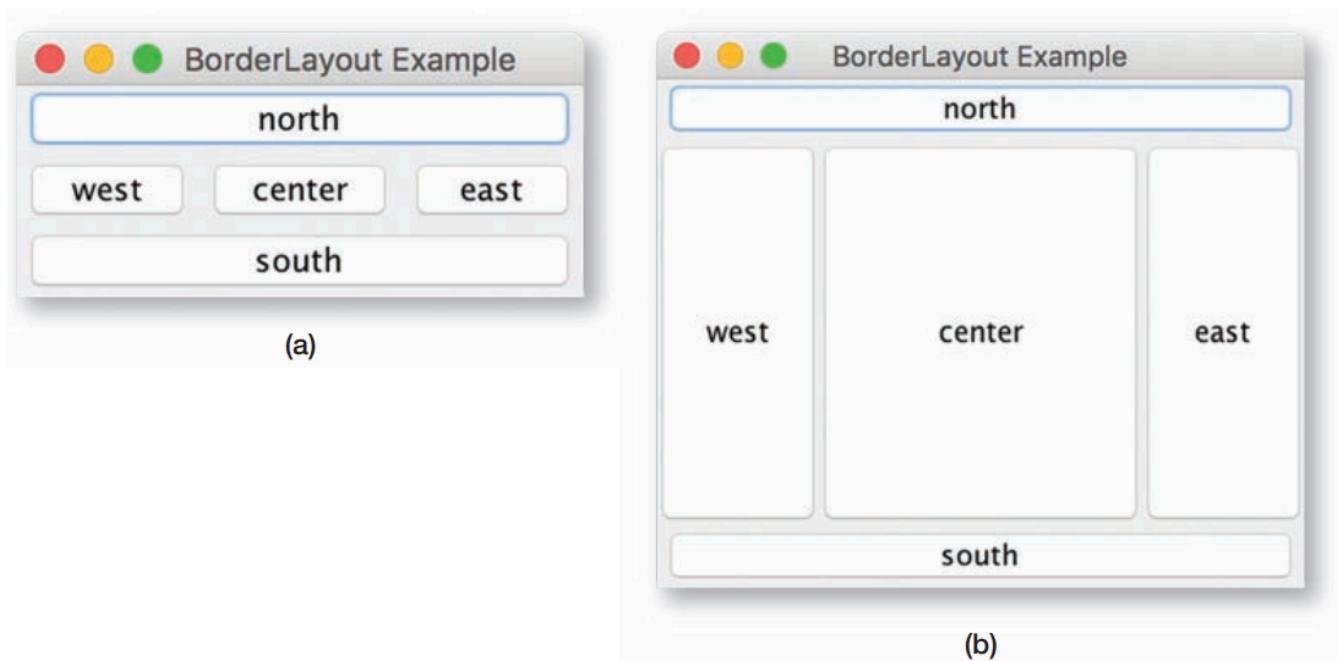
- Useful for simple, sequential arrangements but not ideal for complex grids or precise positioning.

10.4.3. BorderLayout

Definition

BorderLayout divides the container into five regions (North, South, East, West, Center) and places one component in each, stretching them to fit the region.

Features



- Default layout for JFrame's content pane.
- Center region expands to fill remaining space.
- Regions can be empty; components in North/South have fixed height, East/West fixed width.

Example

- In example 1.1, the borderPanel uses:

```
JPanel borderPanel = new JPanel(new BorderLayout());  
borderPanel.add(new JLabel("Center Label"), BorderLayout.CENTER);  
borderPanel.add(new JButton("South Button"), BorderLayout.SOUTH);
```

Remark

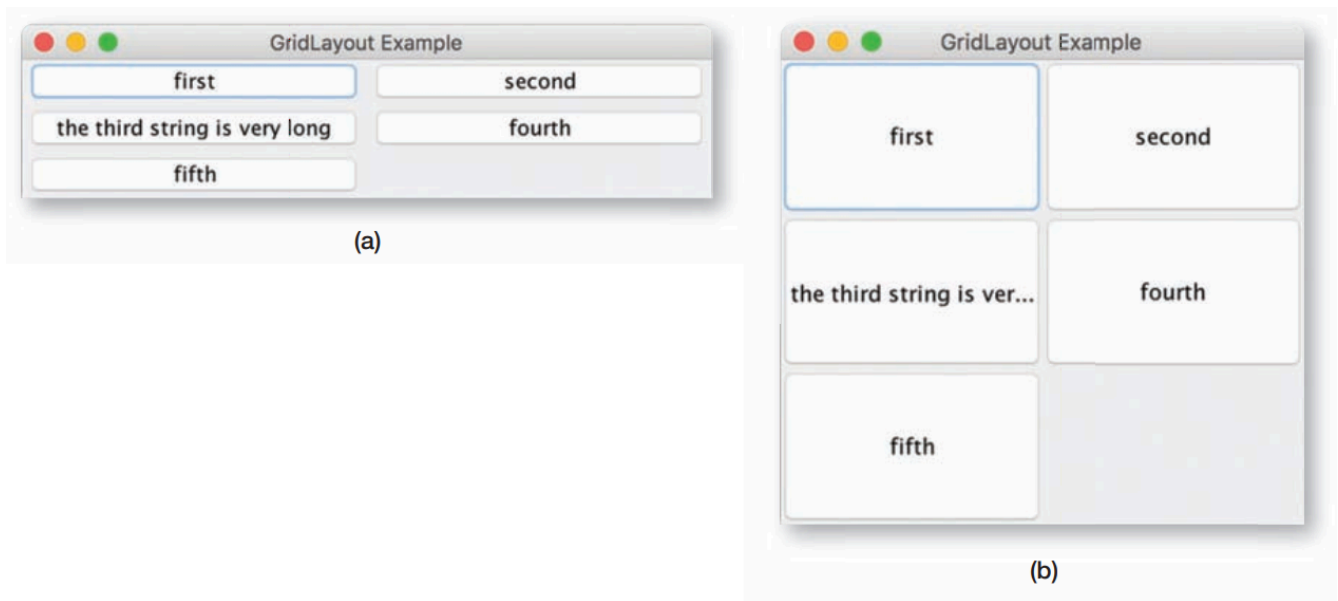
- Great for main window structures (e.g., toolbar North, status bar South), but limited to five components max.

10.4.4. GridLayout

Definition

GridLayout arranges components in a grid of rows and columns, where each cell is the same size, and components are stretched to fill their cells.

Features



- Specified by number of rows and columns (e.g., `new GridLayout(2, 2)`).
- Fills left-to-right, top-to-bottom.
- All components are resized equally, ignoring preferred sizes.

Example

- In example 1.1, the `gridPanel` uses:

```
JPanel gridPanel = new JPanel(new GridLayout(2, 2));
gridPanel.add(new JButton("Grid 1"));
gridPanel.add(new JButton("Grid 2"));
// Additional buttons...
```

Remark

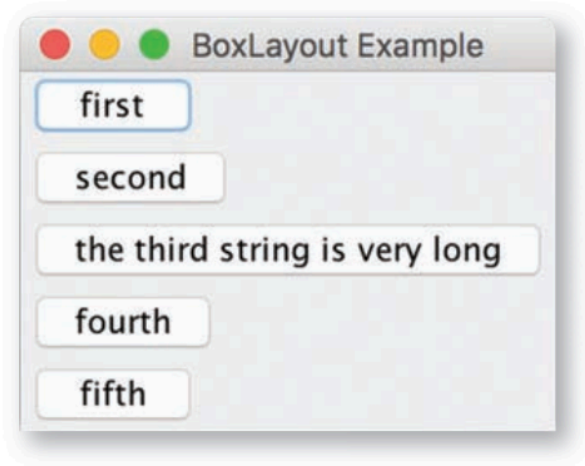
- Ideal for uniform grids like calculators or tables, but can distort components if sizes vary.

10.4.5. BorderLayout

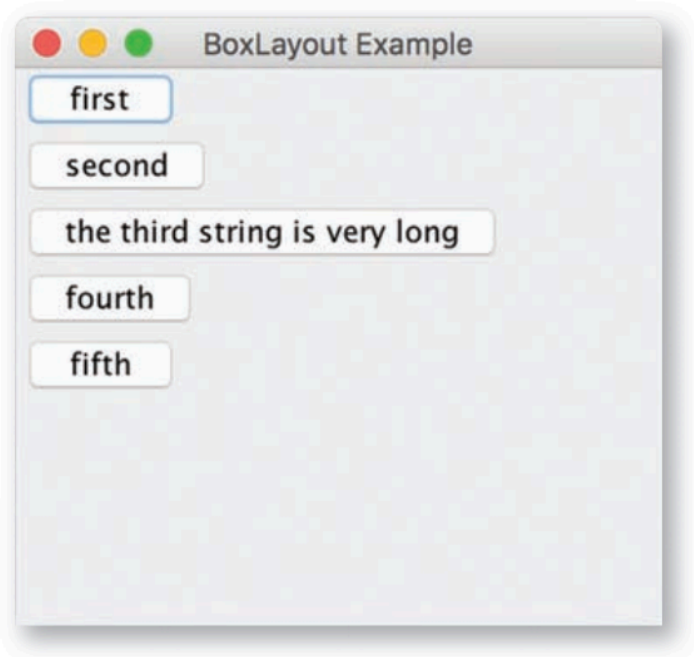
Definition

BoxLayout arranges components in a single row (horizontal) or column (vertical), stacking them sequentially while respecting their preferred sizes.

Features



(a)



(b)

- More flexible than FlowLayout; allows alignment and glue/spacers.
- Direction specified as BoxLayout.X_AXIS or BoxLayout.Y_AXIS.
- Components can be aligned (left, center, right for horizontal; top, center, bottom for vertical).

Example

- In example 1.1, the boxPanel uses:

```
JPanel boxPanel = new JPanel();
boxPanel.setLayout(new BoxLayout(boxPanel, BoxLayout.Y_AXIS));
boxPanel.add(new JLabel("Box Item 1"));
boxPanel.add(Box.createVerticalStrut(10)); // Spacer
boxPanel.add(new JLabel("Box Item 2"));
```

Remark

- Often used with Box class for adding invisible spacers (e.g., createHorizontalGlue() for flexible space); good for toolbars or forms.

The key to building good looking and well-behaved interfaces lies in one last detail: layouts can be nested.

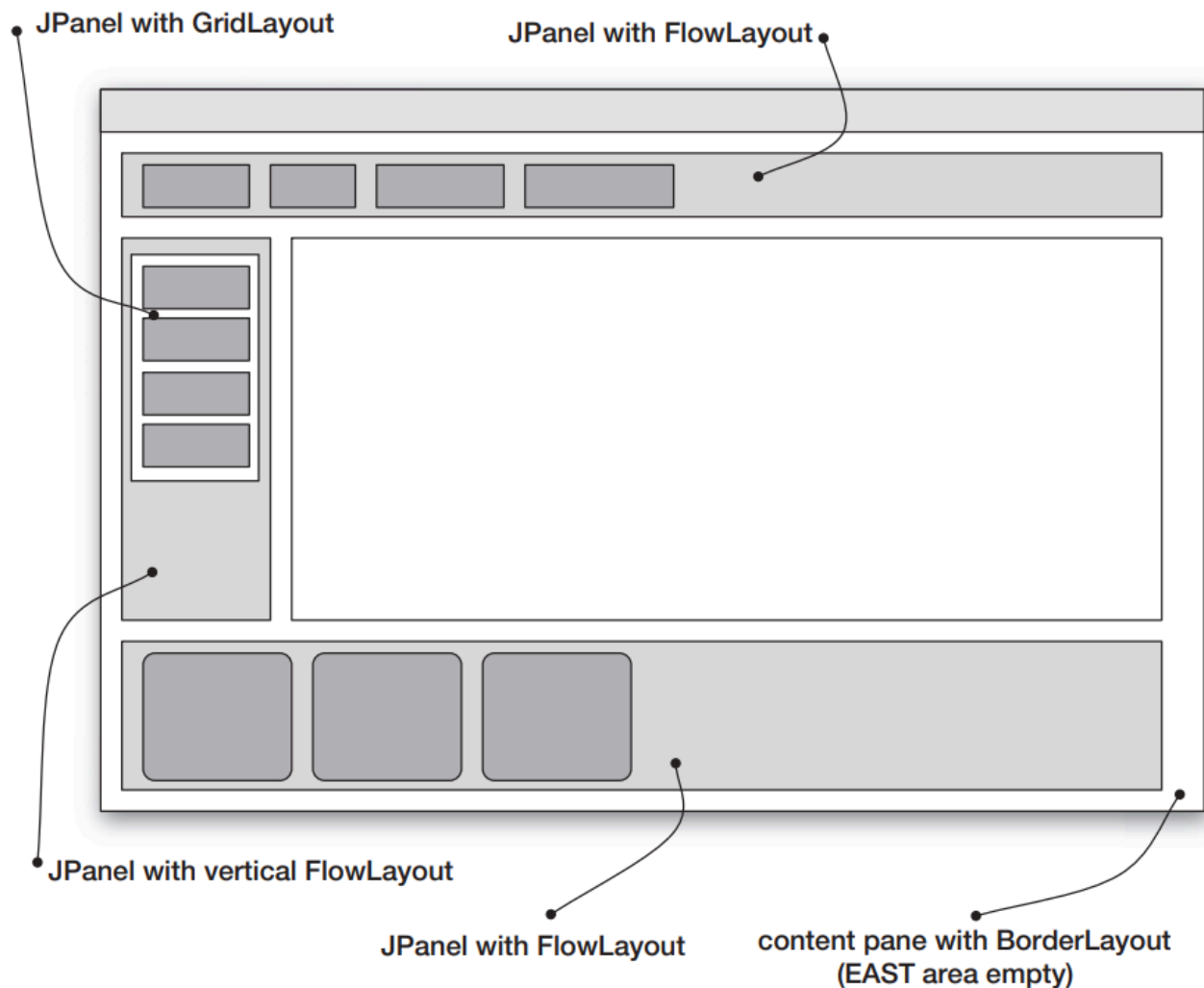
10.5. Container

Definition

Containers appear to the outside as a single component, but they can contain multiple other components. Each container has its own layout manager attached.

Feature

Figure 10.X, for example, shows an interface arrangement similar to that of the BlueJ main window.



- The content pane of this frame uses a `BorderLayout`, where the EAST position is unused.
- The NORTH area of this `BorderLayout` contains a `JPanel` with a horizontal `FlowLayout` that arranges its components (say toolbar buttons) in a row.
- The SOUTH area is similar: another `JPanel` with a `FlowLayout`.

Explanation

The button group in the WEST area was first placed into a `JPanel` with a one-column `GridLayout` to give all buttons the same width. This `JPanel` was then placed into another `JPanel` with a `FlowLayout` so that the grid did not extend over the full height of the WEST area. The outer `JPanel` is then inserted into the WEST area of the frame.

Note

- The most-used container is the class `JPanel`. A `JPanel` can be inserted as a component into the frame's content pane, and then more components can be laid out inside the `JPanel`.

10.6. Event Handling

Definition

ActionEvent is an event class in AWT/Swing that represents a user action, such as clicking a button or selecting a menu item, used in event handling.

Features

- Generated by components like JButton or JMenuItem.
- Passed to ActionListener's `actionPerformed()` method.
- Provides details like the source component via `getSource()` .

Example

- In example 1.1, ActionEvent is handled in:

```
public void actionPerformed(ActionEvent e) {  
    if (e.getSource() == button) {  
        // ...  
    }  
}
```

Remark

- In example 1.1, it's used to distinguish between different sources (button vs. menu item), enabling a single listener to handle multiple events.